

Convolutional Neural Networks for Image Classification: Theory and an Applied Case Study

Prithiv Ramvasan Vetri Selvan

B.Eng. Mechatronic Systems Engineering

SRH Berlin University of Applied Sciences

Module: Artificial Intelligence / Machine Learning (BST-AM)

Examiner: Grigory Devadze

Matriculation number: 3121993

August 27, 2026

Abstract

Convolutional networks are compared against a fully connected baseline on four-class brain-tumour MRI classification (7,200 images). A 25.2 M-parameter MLP reaches 64.4% test accuracy; a 1.29 M-parameter CNN — 19.5× smaller — reaches 86.2%, and a justified augmentation policy lifts this to 91.2%. Auditing the supplied split, however, revealed that 27.7% of test images have a near-identical counterpart in training, invisible to a byte-level duplicate check because the copies were re-encoded. On the 1,157 genuinely held-out images the augmented CNN scores 90.2% accuracy and 0.866 macro F1, and this is the figure reported. Deployment is not recommended: glioma recall of 0.784 misses one in five.

1 Introduction

Medical image classification is a natural test of the claims made for convolutional architectures. Diagnostic features are local, appear at positions that vary between patients, and carry meaning through spatial arrangement rather than absolute pixel coordinates — precisely the structure a convolution encodes and a fully connected layer must learn from scratch.

The task chosen is four-class brain-tumour classification from MRI: glioma, meningioma, pituitary tumour, and no tumour. The dataset is public, is neither MNIST nor FashionMNIST, and was not used in class. It is large enough that the architectural comparison is not noise-dominated and small enough that overfitting is a live concern rather than a theoretical one, which makes the regularisation discussion of Section 3.2 measurable rather than asserted.

Sections 2 and 3 answer the two theoretical questions; Sections 4–7 cover the dataset and an audit of its split, the three models, the comparison isolating architecture and augmentation separately, the failure modes, and a deployment recommendation. Every number is produced by the accompanying code and written to `results/metrics.json`; none is transcribed by hand.

2 Theory I: The Building Blocks of a CNN

A CNN assumes its input lies on a regular grid whose neighbouring elements are related. For images this is true enough to buy a large cut in parameters and a large gain in generalisation. This section follows one 128×128 RGB image through the network in Figure 1, sized for the dataset used in Section 4.

Two patterns in Table 1 matter:

- Spatial size falls while depth rises — the network trades *where* information is for *what* it is.
- Parameters are very unevenly spread: four convolutional layers hold 19% of the weights, while one dense layer holds 81%.

2.1 Convolutional Layers

A convolutional layer slides learnable *kernels* across its input, computing a dot product at each position [5]:

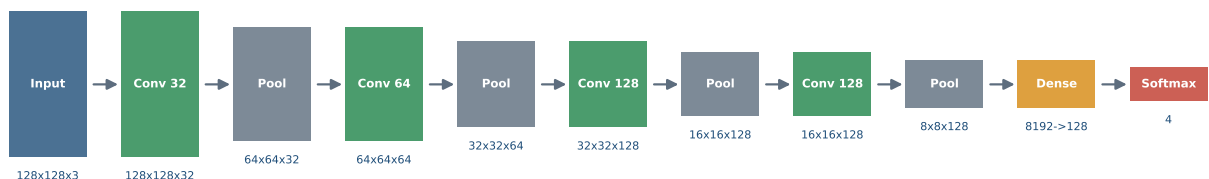
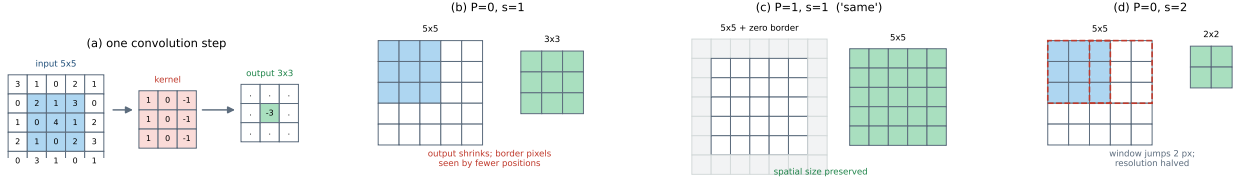
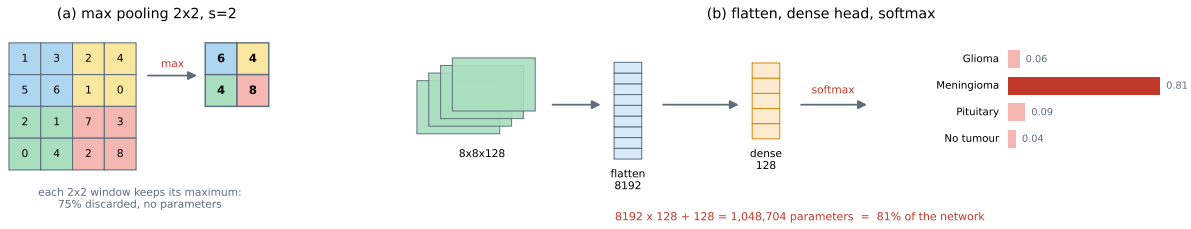


Figure 1: Forward path of the reference network.

Table 1: Layer-by-layer trace of a $128 \times 128 \times 3$ input. All convolutions are 3×3 with $s = 1$ and *same* padding.

#	Layer	Output	Params	#	Layer	Output	Params
0	Input	$128^2 \times 3$	0	6	MaxPool	$16^2 \times 128$	0
1	Conv, 32	$128^2 \times 32$	896	7	Conv, 128	$16^2 \times 128$	147,584
2	MaxPool	$64^2 \times 32$	0	8	MaxPool	$8^2 \times 128$	0
3	Conv, 64	$64^2 \times 64$	18,496	9	Flatten	8192	0
4	MaxPool	$32^2 \times 64$	0	10	Dense, 128	128	1,048,704
5	Conv, 128	$32^2 \times 128$	73,856	11	Dense, 4	4	516
Total: 1,290,052 parameters							

**Figure 2:** (a) One convolution step: patch $\times$ kernel, summed, giving $(2-3) + (0-1) + (1-2) = -3$. (b)–(d) Effect of padding P and stride s on a 5×5 input with a 3×3 kernel.**Figure 3:** (a) Max pooling, 2×2 , stride 2. (b) Flatten, dense layer, softmax.

$$y_{i,j,m} = b_m + \sum_{c=1}^{C_{in}} \sum_{u=1}^K \sum_{v=1}^K w_{u,v,c,m} x_{i \cdot s + u, j \cdot s + v, c} \quad (1)$$

This sum is the *preactivation* z ; applying $\text{ReLU}(z) = \max(0, z)$ gives the *activation*.

The kernel in Figure 2(a), with columns $[1, 0, -1]$, fires where intensity falls left to right — a vertical edge detector. Nobody designed it; it emerges because edges reduce the loss.

Kernel size K

How much one neuron sees. Two stacked 3×3 layers match one 5×5 layer using $18C^2$ weights instead of $25C^2$, plus an extra non-linearity.

Stride s Step size of the window. $s = 2$ halves resolution directly (Figure 2d).

Padding P

Zero border. Without it the map shrinks and border pixels are visited from fewer positions (Figure 2b). *Same* padding, $P = (K - 1)/2$, avoids both (Figure 2c).

Channels C_{out}

Number of kernels, hence output depth. Each kernel spans *all* input channels, so layer 1 holds $(3 \cdot 3 \cdot 3 + 1) \times 32 = 896$ weights.

Output size follows from these:

$$H_{out} = \left\lfloor \frac{H_{in} - K + 2P}{s} \right\rfloor + 1 \quad (2)$$

Note what the 896 excludes — the image resolution. That is the source of the advantage in Section 2.3.

Applications. Any grid-structured signal: RGB images, multispectral satellite bands, medical volumes, and with 1-D kernels, time series such as ECG traces. AlexNet, VGG and ResNet are built from exactly these parts.

2.2 Pooling and the Classification Head

Pooling replaces each neighbourhood with a summary statistic. Max pooling keeps the largest activation and discards the rest (Figure 3a). It has no parameters and does three things:

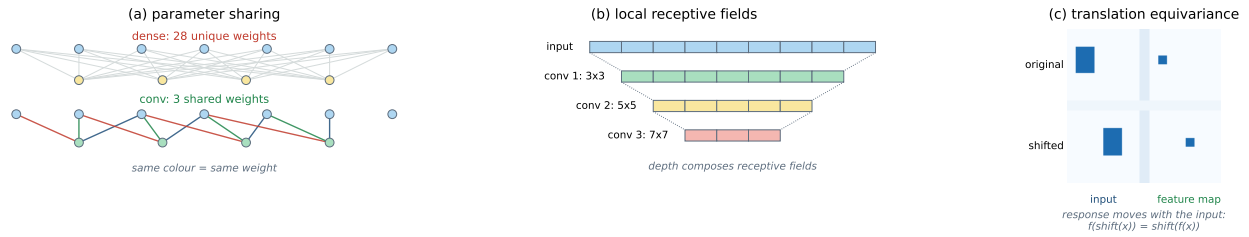


Figure 4: (a) One kernel reused everywhere. (b) Receptive fields composing with depth. (c) A response that moves with its input.

- Cuts the activation volume fourfold — stage 2 of Table 1 reduces 524,288 activations to 131,072.
- Doubles the image area later kernels cover, reaching global context in fewer layers.
- Gives local translation invariance: a one-pixel shift inside a window leaves the maximum unchanged.

The cost is exact position, which suits classification but harms segmentation.

The head flattens the final volume and passes it through dense layers, each computing $\mathbf{Z} = \mathbf{W}\mathbf{X} + \mathbf{b}$, with one output unit per class. Softmax gives $p(y = k | \mathbf{x}) = e^{z_k} / \sum_j e^{z_j}$, and training minimises categorical cross-entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log(p_{ij}) \quad (3)$$

So the conv stack is a learned *feature extractor* and the head is a *classifier*. Flattening $8 \times 8 \times 128$ into 128 units costs 1,048,704 weights — 81% of the network. Two consequences:

- Overfitting originates here, so dropout is applied here (Section 3.2).
- Stack and head are separable, so a pretrained stack can be reused with a fresh head — transfer learning.

2.3 Why CNNs Outperform Fully Connected Networks

A dense network can in principle represent anything a CNN can. The CNN's advantage is *inductive bias*: assumptions true of natural images, so a smaller and better-aimed search space.

Parameter sharing. Dense layers give every connection a private weight; convolution reuses one kernel everywhere. On this input a dense layer of 512 units needs $128 \cdot 128 \cdot 3 \cdot 512 + 512 = 25,166,336$ parameters; the convolutional layer that opens the network needs 896 — about 0.004% as many. Fewer weights means less capacity to memorise and a stronger gradient signal per weight.

Local receptive fields. Each neuron connects only to a small neighbourhood, matching a real property of images: pixel correlation decays with distance. A dense layer must instead *learn* that distant pixels are irrelevant. Long-range structure survives because receptive fields compose with depth — three stacked 3×3 layers give a 7×7 view.

Translation equivariance. Convolution commutes with translation: $f(T_\delta \mathbf{x}) = T_\delta f(\mathbf{x})$. A detector learned in one region works everywhere; a dense network must relearn a pattern for each location. CNNs are often called translation *invariant*. Strictly, convolution is equivariant — the response moves with the input. Invariance belongs to the architecture as a whole, from equivariant convolutions plus pooling discarding exact position.

Together these place the CNN at a better point on the bias–variance trade-off: more constrained, with constraints correct for the domain.

3 Theory II: Training and Generalisation

3.1 Reading Loss Curves

Training loss is measured on the data the optimiser updates on; validation loss on held-out data producing no gradient updates [6].

Underfitting	Both losses flatten high, small gap. The model fails on data it has already seen, so more data will not help. Logistic regression on raw CIFAR-10 pixels stalls near 40%: a linear boundary in pixel space cannot separate the classes. Fix: more capacity, longer schedule, higher learning rate, weaker regularisation.
Good fit	Both fall, validation flattens slightly above training. A small gap is expected; what matters is that validation is flat, not rising.
Overfitting	Training loss $\rightarrow 0$ while validation bottoms out and climbs. The widening gap is the signature. Past the minimum the network memorises sensor noise, compression artefacts and incidental backgrounds. Typical trigger: a high-capacity CNN on a few thousand images, no augmentation.

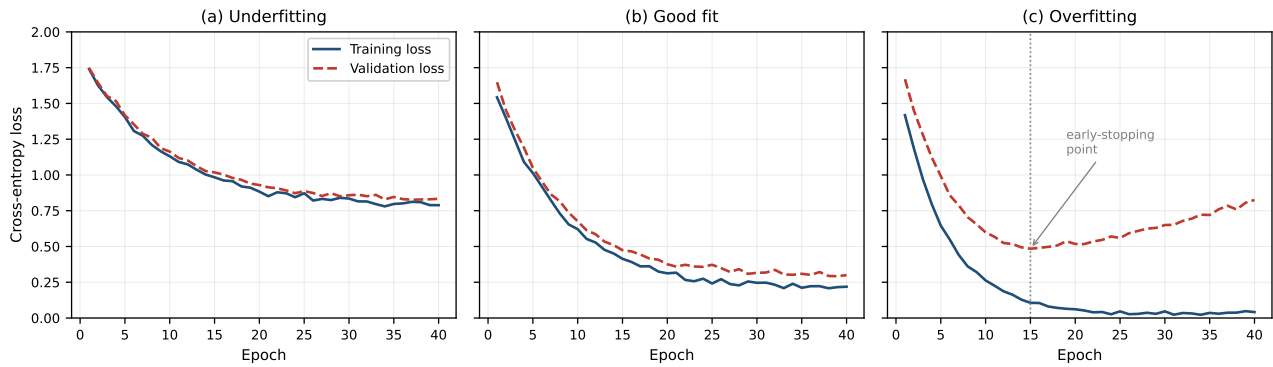


Figure 5: The gap between curves measures variance; the height at which they settle measures bias.

Table 2: Regularisation methods and their mechanisms.

Method	Mechanism
Dropout	Randomly deactivates neurons during training
Batch normalisation	Normalises activations per mini-batch
Layer normalisation	Normalises across features; used in sequence models
Data augmentation	Expands training data with label-preserving transforms
Weight decay (L_2)	Penalises large weight magnitudes
Early stopping	Halts training at the validation minimum

Caution: validation loss *below* training loss usually signals a bug — often dropout active in training but not validation.

3.2 Dropout, Augmentation, Batch Normalisation

Dropout zeroes each unit with probability p (0.1–0.5) every forward pass, scaling survivors by $1/(1-p)$; at inference it is off. It attacks *co-adaptation* — units forming fragile committees where one is useful only because three specific others fire alongside it. Since no unit can rely on a partner, each must learn an independently informative feature. Equivalently, n units define 2^n sub-networks sharing one weight set. Applied in the dense head, where parameters concentrate.

Data augmentation applies label-preserving transforms on the fly. The obvious effect is a larger training set; the important one is that it *states an invariance* the architecture does not encode. The transform set is a modelling decision, not a default:

- Satellite imagery has no canonical orientation — flips and 90° rotations are valid, giving eight free variants per image.
- Traffic signs are the opposite: a vertical flip turns a left-turn arrow into a right-turn arrow while keeping the original label.

Batch normalisation normalises each channel per mini-batch, then rescales with two learned parameters:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \quad (4)$$

Layer inputs shift during training because the layers beneath change simultaneously. Fixing the first two moments stabilises this, allowing higher learning rates and reducing sensitivity to initialisation. Standard placement is `Conv` → `BatchNorm` → `ReLU`. At inference, running averages replace batch statistics — forgetting to switch to evaluation mode is a common bug.

3.3 Why a Separate Test Set Is Still Required

The validation set stops being unbiased the moment it is used to make a decision. No validation data enters the gradient computation — but it is not used once. It selects the learning rate, the architecture, whether augmentation helped, the stopping epoch.

The optimiser fits the training set by gradient descent; the developer fits the validation set by repeated selection. The mechanism differs, the consequence does not.

Example. Twenty configurations, each truly 88% accurate, with validation accuracy fluctuating ± 1.5 points because the validation set is finite. Taking the best of twenty reliably returns about 91% — three points that are the maximum of twenty noisy draws, not a better model. The bias grows with the number of configurations tried.

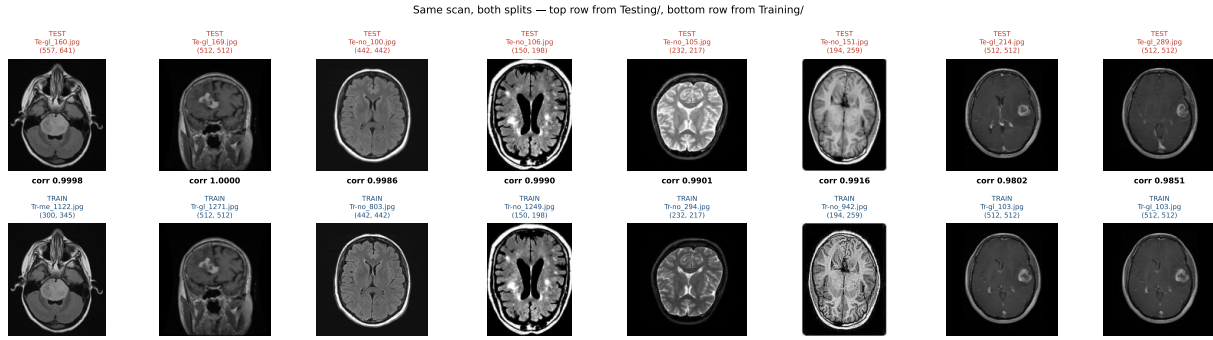


Figure 6: Scans present in both splits: top row from **Testing/**, bottom row from **Training/**, with the correlation between them. The rightmost pair are adjacent slices of one patient rather than one scan duplicated.

The test set is evaluated once, after every design decision is final. If a disappointing test result prompts an architecture change, it has become a validation set and its guarantee is gone. All model selection here uses validation data alone; the test set is untouched until Section 6.

4 Dataset and Preprocessing

Source, licence, size. The four-class Brain Tumour MRI dataset [1] is a public Kaggle compilation of three upstream sources (figshare, SARTAJ, Br35H), and neither MNIST nor FashionMNIST. The copy used here holds 7,200 images balanced at 1,400 per class in **Training/** and 400 in **Testing/** — a resampled variant of the 7,023 unbalanced upstream release, so every count below comes from the copy actually used.

Domain relevance. Gliomas and meningiomas differ mainly in location and margin character, and both can resemble normal tissue at low resolution. The task therefore exercises sensitivity to local structure at an arbitrary position — what convolution supplies — rather than rewarding a memorised global template.

Preprocessing. An inventory pass found 75 distinct image sizes, a dominant 512×512 mode, and a mix of RGB and greyscale files. Scans are cropped to the bounding box of non-background pixels — much of each canvas is black padding — then resized to 128×128 and normalised with ImageNet statistics. **Testing/** is read once, by `evaluate.py`; a stratified 80/20 split of **Training/** gives 4,480 training, 1,120 validation and 1,600 test images, augmentation applying to the training subset only.

4.1 An audit of the supplied split

The guarantee a held-out set provides (Section 3.3) is a property of the split, not of the folder names, so it was checked. Kapoor and Narayanan [8] survey 294 papers across 17 fields and find leakage of this kind to be a leading cause of irreproducible results, which is reason enough not to take a shipped split on trust. Hashing *file bytes* reports zero duplicates spanning the two folders; hashing the *decoded pixels* gives a different answer, since the byte hashes differ only because the copies were re-encoded. 114 test images are pixel-identical to a training image, verified at full resolution ($\max |\Delta| = 0$). Extending this to near-duplicates by normalised cross-correlation of 64×64 thumbnails — invariant to brightness and contrast — flags 443 of 1,600 test images (27.7%): 333 *notumor* (83.2% of the class), 103 *meningioma*, 7 *glioma*, 0 *pituitary*. Pituitary scoring zero is the control that makes the measurement credible: the metric fires only where duplicated scans exist, not on generic MRI similarity.

Two further defects surfaced: two flagged pairs carry *conflicting labels* (one scan is glioma in **Testing/** and meningioma in **Training/** at correlation 0.9998, so at least one is wrong), and several pairs are adjacent slices of one patient split across folders — the patient-level leakage a dataset without patient identifiers cannot be filtered for. All results below are therefore reported over the full test set *and* over the 1,157 images with no training counterpart; the second is the defensible figure. A smaller instance affects validation, where 187 redundant copies within **Training/** leave 59 of 1,120 validation images (5.27%) byte-identical to a training image.

5 Methodology

All three configurations share Adam at 10^{-3} , batch size 32, cross-entropy loss, seed 42, and early stopping on validation loss with patience 8; only architecture and augmentation vary, so each comparison isolates one factor.

Baseline. A multilayer perceptron: flatten, one hidden layer of 512 units with ReLU, dropout 0.5, linear classifier — 25,168,388 parameters, 25.2 M in the first layer alone. Flattening destroys spatial adjacency before the first weight is applied, so it has no locality, no parameter sharing and no translation equivariance: the three properties of Section 2.3. Any gap is therefore attributable to those rather than to capacity, the baseline carrying nineteen times more parameters than the model that beats it.

Improved CNN. Four blocks of Conv–BatchNorm–ReLU–MaxPool with 32, 64, 128 and 128 channels, then a dense head of 128 units with dropout 0.5; 1,290,404 parameters. 3×3 *kernels throughout* [4]: two stacked 3×3 layers span the same 5×5 receptive field using $18C^2$ weights rather than $25C^2$, with an extra non-linearity

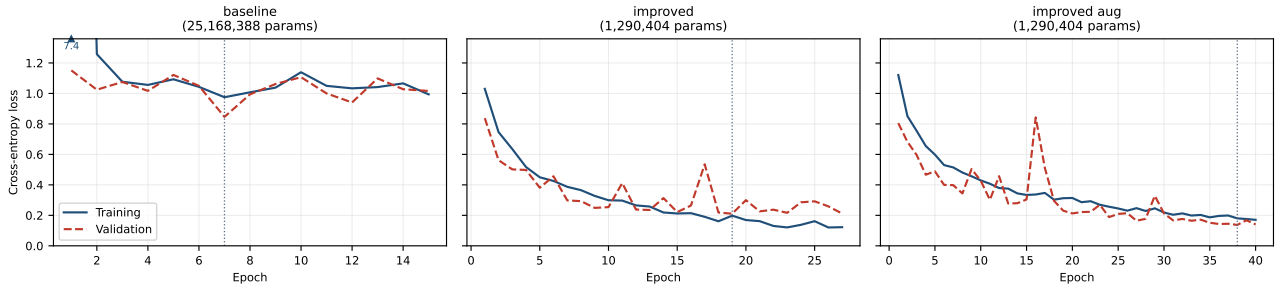


Figure 7: Training and validation cross-entropy; dotted line marks the best-validation epoch. The baseline’s first-epoch loss of 7.41 is off-scale, marked with a triangle.

Table 3: Model comparison. *Clean* is the 1,157 test images with no training counterpart (Section 4.1); macro F1 follows the same column.

Configuration	Parameters	Epochs	Val. acc.	Test accuracy		
				Full	Clean	Macro F1
Baseline (MLP)	25,168,388	15/30	0.688	0.644	—	0.641
Improved CNN	1,290,404	27/30	0.921	0.862	—	0.860
+ augmentation	1,290,404	40/40	0.952	0.912	0.902	0.866

between them. *Same padding*, so border pixels are sampled as often as central ones and downsampling happens only in the pooling layers, where it is explicit. *Channels double as resolution halves*, holding representational capacity roughly constant while features grow more abstract. *BatchNorm before every activation* [2], so the non-linearity operates in a consistent range across batches; since it supplies its own shift, the preceding convolutions are declared `bias=False`. *Dropout only in the head* [3], which holds 81.3% of the parameters and is therefore where overfitting originates — convolutional layers are already regularised by weight sharing.

Augmentation. Three transforms, applied to the training subset only, each stating an invariance the architecture does not encode (Section 3.2; see [7] for a survey of the design space): horizontal flip at $p = 0.5$, since tumours occur in either hemisphere; rotation of $\pm 10^\circ$, since head positioning varies between patients while the label does not; and brightness/contrast jitter of $\pm 15\%$, since the three upstream sources used different scanners, making intensity a nuisance variable. Vertical flip is excluded — anatomically implausible, it would assert an invariance that is false. The effect is measured in Section 6.2, not assumed.

6 Results and Discussion

6.1 Learning-Curve Analysis

The three panels correspond to the three regimes of Section 3. The baseline **underfits**: both curves flatten near a cross-entropy of 1.0 with a small gap and stay there, early stopping halting it at epoch 15. Both losses high together is the diagnostic — the model fails on data it has already seen, so more data would not help. It is not capacity-limited, having $19.5\times$ more parameters than the model that beats it; it is structurally unsuited to the task. The un-augmented CNN begins to **overfit**: training loss falls to 0.13 while validation stalls near 0.21 and turns erratic after epoch 20, the widening gap being the signature. The augmented CNN shows a **good fit**: the curves track each other throughout, validation sitting at or slightly below training — consistent with augmentation applying to the training subset alone, so training loss is measured on a harder distribution. That run exhausted its 40-epoch budget with its best epoch at 38, never triggering early stopping, so 91.2% is a floor for the configuration rather than its ceiling.

6.2 Quantitative Comparison

Chance level for four balanced classes is 0.25. **Architecture:** the CNN improves test accuracy by 21.7 points ($0.644 \rightarrow 0.862$) using $19.5\times$ fewer parameters. Optimiser, schedule, data and augmentation are identical across the two runs, so the difference is attributable to inductive bias alone — the concrete form of the argument in Section 2.3. **Augmentation:** holding architecture fixed, the policy of Section 5 adds 5.0 points of test accuracy ($0.862 \rightarrow 0.912$) and 3.0 of validation accuracy, its regularising effect visible in Figure 7 as the closed gap between curves — the mechanism, not merely the outcome.

Excluding duplicated test images costs 1.0 point of accuracy ($0.912 \rightarrow 0.902$) but 4.4 of macro F1 ($0.910 \rightarrow 0.866$), the contamination sitting almost entirely in one class: only 67 of 400 *notumor* test images survive the filter, and on those its precision falls from 0.846 to 0.568. Its perfect recall over the full test set is largely recall on images already seen, and the class is effectively untestable here. On the clean subset glioma scores $P0.981/R0.784/F10.871$ ($n = 393$), meningioma $0.874/0.912/0.893$ ($n = 297$) and pituitary $0.959/0.995/0.977$ ($n = 400$); the latter two barely move between columns, having almost no contaminated images.

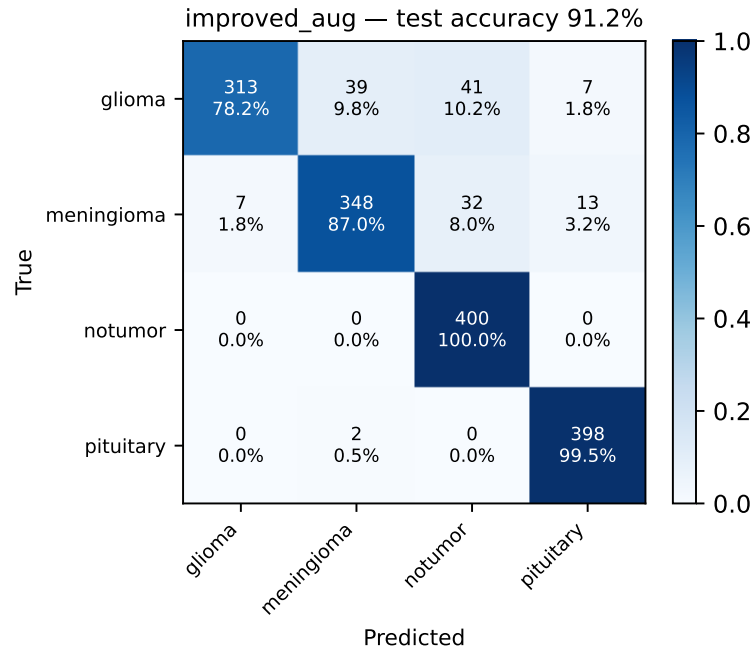


Figure 8: Confusion matrix over the full test set; rows are true classes. The *notumor* column absorbs 73 scans from the two tumour classes, which is the whole of its precision deficit.

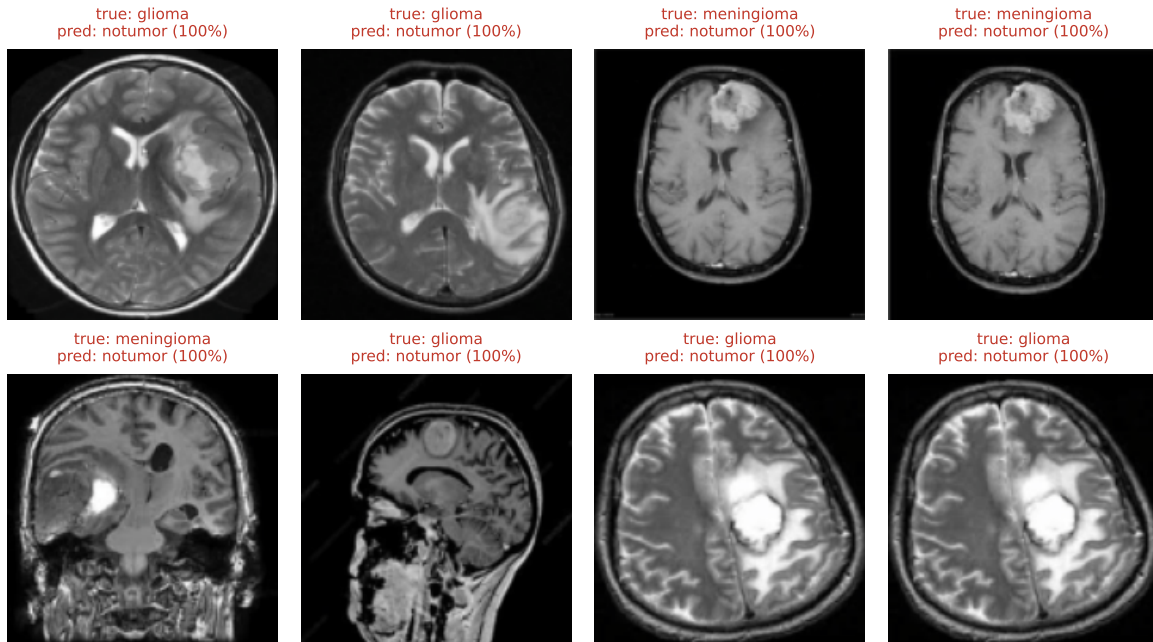


Figure 9: The eight most confident misclassifications. Six are gliomas assigned to *notumor* at 100% confidence, several of them showing a large and visibly hyperintense lesion.

6.3 Failure Cases and Improvement Ideas

Of 141 errors, 87 are gliomas, splitting almost evenly between *notumor* (41) and *meningioma* (39). *Notumor* acts as a sink: recall 1.000, but the worst precision of the four classes, with 73 scans from the tumour classes pulled into it. The direction matters clinically — 41 of 400 gliomas (10.2%) are called healthy while the reverse error never occurs. A model that never mistakes a healthy scan for a tumour but discards one glioma in ten is optimised for the wrong asymmetry.

Figure 9 sharpens this: six of the eight most confident errors are gliomas predicted *notumor* at 100% confidence, several showing a large, visibly hyperintense lesion. A merely weak model produces *uncertain* errors; total confidence on an obvious lesion indicates either a region of the input distribution never learned, or mislabelled source data — plausible here, since the upstream compilers documented replacing an unreliable SARTAJ glioma folder and Section 4.1 already found one provably wrong label in this copy.

Four improvements follow, in order of expected return. *Fix the split first:* any gain measured on a 27.7%-

contaminated test set is unverifiable, so deduplicate at pixel level and re-split. *Weight the loss towards glioma recall*, or lower its decision threshold, trading notumor precision for the error direction that matters at no architectural cost. *Audit the glioma labels*: if the confident failures are mislabelled, achievable recall exceeds 0.784. *Transfer learning*: an ImageNet-pretrained backbone fine-tuned here would likely beat a from-scratch 1.29 M network on 4,480 images, at the cost of the controlled comparison this report is built around.

7 Conclusion and Deployment Recommendation

Both hypotheses held under controlled comparison: the CNN outperformed a fully connected baseline by 21.7 points of test accuracy using 19.5× fewer parameters, and augmentation added 5.0 points more while visibly closing the generalisation gap. The augmented CNN is the model to carry forward.

It should not be deployed. Three findings independently block that. Glioma recall of 0.784 means roughly one glioma in five is missed, and the errors run in the clinically unsafe direction — towards *notumor*, never away from it. The test split is 27.7% contaminated, so even the corrected 90.2% rests on a dataset that cannot be fully audited and contains at least one provably wrong label. And with no patient identifiers a patient-wise split is impossible, so some optimism remains in any figure computed from it.

The defensible claim is narrower: on this data the convolutional inductive bias and a modest augmentation policy together account for a 26.7-point improvement over a far larger fully connected model. That is a statement about architecture, which is what the experiment tested — not about clinical readiness, which would need a patient-wise split, a verified label set, and a threshold tuned for the asymmetric cost of a missed tumour.

Reproducibility

Seeds are fixed at 42 across Python, NumPy and PyTorch, with cuDNN deterministic. Every figure and number in Sections 4 and 6 is generated by the accompanying code and written to `results/metrics.json` and `results/figures/`; none is transcribed by hand. The test set is read once, by `evaluate.py`, after all three models were trained. Runs used CPU (2 cores), PyTorch 2.13.0, torchvision 0.28.0 and Python 3.11, pinned in `requirements-lock.txt`; total training time was 6,980 s. The pipeline reproduces by running `notebooks/brain_tumour_cnn.ipynb` top to bottom.

Appendix A: Independent reproduction

Console output from running `notebooks/brain_tumour_cnn.ipynb` end to end on a second machine (Windows, Python 3.13, CPU) from the shipped checkpoints; every quoted figure reproduces to four decimal places. The MLP baseline is absent only because its checkpoint is 96 MB and is regenerated rather than shipped.

```
python 3.13.0 torch 2.13.0+cpu torchvision 0.28.0+cpu numpy 2.5.1 device cpu
scikit-learn 1.9.0 matplotlib 3.11.1 pillow 12.3.0
```

config	parameters	val acc	test acc	macro F1	(chance = 0.2500)
improved	1,290,404	0.9214	0.8619	0.8599	
improved_aug	1,290,404	0.9518	0.9119	0.9103	

	n	accuracy	macro F1	
improved_aug				
full test set	1600	0.9119	0.9103	
contaminated	443	0.9368		
CLEAN (no train twin)	1157	0.9023	0.8663	(contamination 27.7%)

References

- [1] Nickparvar, M. (2021). *Brain Tumor MRI Dataset*. Kaggle; compiled from figshare, SARTAJ, Br35H. kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset
- [2] Ioffe, S. & Szegedy, C. (2015). Batch normalization. *ICML*.
- [3] Srivastava, N. et al. (2014). Dropout. *JMLR* 15(1), 1929–1958.
- [4] Simonyan, K. & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *ICLR*.
- [5] LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86(11), 2278–2324.
- [6] Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press.
- [7] Shorten, C. & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data* 6, article 60.
- [8] Kapoor, S. & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 4(9), 100804.